# Home Assignment 3

Roque Hauser

2026-11-11

## Contents

**Exercise 1** **1**

**Exercise 2** **4**

**Exercise 3** **6**

**Exercise 4** **8**

**Exercise 5** **11**

**Exercise 6** **13**

View RStudio Project on GitHub

## Exercise 1

### Question

Calculate the value-weighted portfolio return as in HA2 Q6, now subject to a maximum weight of 35% in any single asset. Briefly explain how you enforce the weight limit. What is the mean realised return per annum?

### Answer

The portfolio weights are based on the previous month's market capitalisations and capped at 35%, with excess weight redistributed proportionally among the remaining stocks.

The mean annualised return is **29.0984933%**.

Therefore, the correct answer is **c. 27–30%**.

### R Code

```r
# Exercise 1

# Data preparation for Exercise 1
data_1 <- monthly_cleaned %>%
  select(Date, all_of(tri_cols), all_of(cap_cols)) %>%
  arrange(Date)

returns_1 <- data_1 %>%
  select(all_of(tri_cols)) %>%
  mutate(across(everything(), ~ .x / lag(.x) - 1))

caps_1 <- data_1 %>%
  select(all_of(cap_cols)) %>%
  mutate(across(everything(), ~ lag(.x)))

portfolio_data_1 <- data.frame(
  Date = data_1$Date,
  returns_1,
  caps_1
) %>%
  filter(
    Date >= as.Date("2011-01-01"),
    Date <= as.Date("2021-12-31")
  )

# Portfolio calculations
cap_weights_1 <- function(caps_1, max_weight_1 = 0.35) {

  weights_1 <- numeric(length(caps_1))
  remaining_1 <- seq_along(caps_1)

  while (length(remaining_1) > 0) {
    available_weight_1 <- 1 - sum(weights_1)
    new_weights_1 <- available_weight_1 *
      caps_1[remaining_1] / sum(caps_1[remaining_1])
    over_limit_1 <- new_weights_1 > max_weight_1

    if (!any(over_limit_1)) {
      weights_1[remaining_1] <- new_weights_1
      break
    }

    weights_1[remaining_1[over_limit_1]] <- max_weight_1
    remaining_1 <- remaining_1[!over_limit_1]
  }

  return(weights_1)
}

# Calculate monthly portfolio returns
portfolio_returns_1 <- numeric(nrow(portfolio_data_1))
portfolio_weights_1 <- matrix(
  NA_real_,
```

```r
  nrow = nrow(portfolio_data_1),
  ncol = length(stocks)
)

for (i in seq_len(nrow(portfolio_data_1))) {

  stock_returns_1 <- as.numeric(portfolio_data_1[i, 2:6])

  market_caps_1 <- as.numeric(portfolio_data_1[i, 7:11])

  weights_1 <- cap_weights_1(market_caps_1)

  portfolio_weights_1[i, ] <- weights_1
  portfolio_returns_1[i] <- sum(weights_1 * stock_returns_1)
}

# Calculate mean return
mean_monthly_return_1 <- mean(portfolio_returns_1)

mean_annual_return_1 <- (
  (1 + mean_monthly_return_1)^12 - 1
) * 100

# Results
#print(mean_monthly_return_1 * 100)
print(mean_annual_return_1)
#print(length(portfolio_returns_1))
```

# Exercise 2

### Question

HA2 Q10 continued. Calculate the global minimum-variance portfolio's weights, expected return, and volatility. Answer: What is its expected return per annum?

### Answer

The global minimum-variance portfolio was calculated using the monthly mean returns and sample covariance matrix of the five stocks.

The portfolio has an expected annualised return of **19.3458588%** and a monthly volatility of **3.8792156%**.

Therefore, the correct answer is **c. 19–20%**.

Table 1: Global Minimum-Variance Portfolio Weights

| Stock | Portfolio Weight (%) |
|---|---|
| Apple | 4.90 |
| Microsoft | 26.72 |
| Tesla | -0.79 |
| JP Morgan Chase | 15.43 |
| Walmart | 53.74 |

### R Code

```
# Exercise 2

# Use monthly returns from Exercise 1
returns_matrix_2 <- as.matrix(
  portfolio_data_1[, 2:6]
)

# Calculation
mean_returns_2 <- colMeans(returns_matrix_2)
Sigma_2 <- cov(returns_matrix_2)

ones_2 <- rep(1, length(mean_returns_2))

weights_gmv_2 <- as.numeric(
  solve(Sigma_2, ones_2) /
    sum(solve(Sigma_2, ones_2))
)

mean_monthly_return_2 <- sum(
  weights_gmv_2 * mean_returns_2
)

volatility_monthly_2 <- sqrt(
  as.numeric(t(weights_gmv_2) %*% Sigma_2 %*% weights_gmv_2)
```

```r
)

mean_annual_return_2 <- (
  (1 + mean_monthly_return_2)^12 - 1
) * 100

# Results
#print(weights_gmv_2)
#print(mean_monthly_return_2 * 100)
#print(volatility_monthly_2 * 100)
print(mean_annual_return_2)
```

## Exercise 3

### Question

Assume that the risk-free rate is given in column AQ. Divide by 100 to convert the annual percentage rate to a decimal, and then divide by 12 to approximate the monthly rate. Use the rate observed at the end of the preceding month: for January 2022, use the value from 31 December 2021; for February, use the value from the end of January; and so on. This ensures that the risk-free return is known before the investment period. Re-estimate the return and volatility of the portfolio on the capital market line with a target return of 1% per month. What is the expected volatility for January 2022?

### Answer

Using historical monthly returns from January 2011 to December 2021 and the risk-free rate from December 2021, the CML portfolio was calculated for a target return of 1% per month.

The expected monthly volatility for January 2022 is **2.1121753%**.

Therefore, the correct answer is **c. 2.0–2.2%**.

### R Code

```r
# Exercise 3

# Use data from Exercise 2
mean_returns_3 <- mean_returns_2
Sigma_3 <- Sigma_2
ones_3 <- rep(1, length(mean_returns_3))

# Find the risk-free rate column (AQ)
rf_col_3 <- grep(
  "US.DOLLAR.*MIDDLE.RATE",
  names(monthly_cleaned),
  value = TRUE
)

rf_annual_3 <- monthly_cleaned %>%
  filter(Date == as.Date("2021-12-31")) %>%
  pull(all_of(rf_col_3))

# Convert annual percentage rate to monthly decimal
rf_monthly_3 <- rf_annual_3 / 100 / 12

target_return_3 <- 0.01

# Calculate A, B, C and K squared
Sigma_inv_R_3 <- solve(Sigma_3, mean_returns_3)
Sigma_inv_ones_3 <- solve(Sigma_3, ones_3)

A_3 <- as.numeric(t(mean_returns_3) %*% Sigma_inv_R_3)
B_3 <- as.numeric(t(ones_3) %*% Sigma_inv_R_3)
C_3 <- as.numeric(t(ones_3) %*% Sigma_inv_ones_3)
```

```r
K2_3 <- A_3 - 2 * B_3 * rf_monthly_3 +
  C_3 * rf_monthly_3^2

# Calculate weights
weights_cml_3 <- as.numeric(
  solve(Sigma_3, mean_returns_3 - rf_monthly_3 * ones_3) *
    (target_return_3 - rf_monthly_3) / K2_3
)

weight_rf_3 <- 1 - sum(weights_cml_3)

# Calculate expected monthly return
expected_return_3 <- sum(weights_cml_3 * mean_returns_3) +
  weight_rf_3 * rf_monthly_3

# Calculate expected monthly volatility
volatility_monthly_3 <- sqrt(
  as.numeric(t(weights_cml_3) %*% Sigma_3 %*% weights_cml_3)
)

# Results
#print(rf_monthly_3 * 100)
#print(weights_cml_3)
#print(weight_rf_3)
#print(expected_return_3 * 100)
print(volatility_monthly_3 * 100)
```

# Exercise 4

### Question

(2 points) Q3 continued. Continue in a rolling fashion for February–October 2022. Each month, add the newly available observation and drop the oldest observation, keeping the estimation-window length fixed. Estimate the expected returns and variance–covariance matrix, then calculate the expected portfolio volatility for each month. Answer: What is the average expected monthly volatility over January–October 2022, including the January estimate from Q3? R hint: use colMeans() and cov() within a rolling-window loop. If calculating the sample covariance matrix directly, let R be the T × N return matrix and rˆT the 1 × N row vector of column means.

### Answer

Using a rolling estimation window of 132 monthly returns, the expected portfolio volatilities were calculated for January to October 2022.

The average expected monthly volatility is **2.1306117%**.

Therefore, the correct answer is **c. 2.0–2.2%**.

Table 2: Rolling Expected Monthly Portfolio Volatility

| Month | Expected Volatility (%) |
|---|---|
| January 2022 | 2.11 |
| February 2022 | 2.18 |
| March 2022 | 2.16 |
| April 2022 | 2.09 |
| May 2022 | 2.16 |
| June 2022 | 2.20 |
| July 2022 | 2.20 |
| August 2022 | 2.08 |
| September 2022 | 2.08 |
| October 2022 | 2.03 |

### R Code

```r
# Exercise 4

# Combine dates and monthly returns from Exercise 1
returns_data_4 <- data.frame(
  Date = data_1$Date,
  returns_1
)

# Investment months: February to October 2022
investment_months_4 <- seq(
  as.Date("2022-02-01"),
  as.Date("2022-10-01"),
  by = "month"
)
```

```r
# Store monthly weights and expected volatilities
portfolio_weights_4 <- matrix(
  NA_real_,
  nrow = length(investment_months_4),
  ncol = length(stocks)
)

rolling_results_4 <- data.frame()

for (i in seq_along(investment_months_4)) {

  # Select the most recent 132 monthly returns
  window_4 <- returns_data_4 %>%
    filter(Date < investment_months_4[i]) %>%
    tail(132)

  # Mean returns and sample covariance matrix
  mean_returns_4 <- colMeans(window_4[, -1])
  Sigma_4 <- cov(window_4[, -1])
  ones_4 <- rep(1, length(mean_returns_4))

  # Risk-free rate
  previous_date_4 <- max(window_4$Date)

  rf_annual_4 <- monthly_cleaned %>%
    filter(Date == previous_date_4) %>%
    pull(all_of(rf_col_3))

  rf_monthly_4 <- rf_annual_4 / 100 / 12

  # Calculate A, B, C and K squared
  Sigma_inv_R_4 <- solve(Sigma_4, mean_returns_4)
  Sigma_inv_ones_4 <- solve(Sigma_4, ones_4)

  A_4 <- as.numeric(t(mean_returns_4) %*% Sigma_inv_R_4)
  B_4 <- as.numeric(t(ones_4) %*% Sigma_inv_R_4)
  C_4 <- as.numeric(t(ones_4) %*% Sigma_inv_ones_4)

  K2_4 <- A_4 - 2 * B_4 * rf_monthly_4 +
    C_4 * rf_monthly_4^2

  # Calculate weights
  weights_4 <- as.numeric(
    solve(Sigma_4, mean_returns_4 - rf_monthly_4 * ones_4) *
      (target_return_3 - rf_monthly_4) / K2_4
  )

  # Calculate expected monthly volatility
  volatility_4 <- sqrt(
    as.numeric(t(weights_4) %*% Sigma_4 %*% weights_4)
  )

  # Store weights and volatility
```

```
  portfolio_weights_4[i, ] <- weights_4

  rolling_results_4 <- rbind(
    rolling_results_4,
    data.frame(
      Date = investment_months_4[i],
      Expected_Volatility = volatility_4
    )
  )
}

# Include January result from Exercise 3
rolling_results_4 <- rbind(
  data.frame(
    Date = as.Date("2022-01-31"),
    Expected_Volatility = volatility_monthly_3
  ),
  rolling_results_4
)

# Include January portfolio weights from Exercise 3
portfolio_weights_4 <- rbind(
  weights_cml_3,
  portfolio_weights_4
)

# Calculate average expected monthly volatility
average_volatility_4 <- mean(
  rolling_results_4$Expected_Volatility
) * 100

# Results
print(rolling_results_4)
print(average_volatility_4)
```

# Exercise 5

### Question

Use the rolling portfolio weights from Q4, including the January weightsf rom Q3, to calculate the realised portfolio return for January–September 2022. Report the cumulative realised return over that period.

### Answer

Using the rolling portfolio weights from Exercises 3 and 4, the realised monthly portfolio returns were calculated for January to September 2022, including the risk-free allocation.

The cumulative realised return over this period is **-8.9843662%**.

Therefore, the correct answer is **a. Less than 15%**.

Table 3: Realised Monthly Portfolio Returns

| Month | Realised Portfolio Return (%) |
|---|---|
| January 2022 | -2.56 |
| February 2022 | -1.87 |
| March 2022 | 3.63 |
| April 2022 | -2.93 |
| May 2022 | -3.69 |
| June 2022 | -2.69 |
| July 2022 | 5.55 |
| August 2022 | -1.60 |
| September 2022 | -2.79 |

### R Code

```
# Exercise 5

# Select realised monthly stock returns for January-September 2022
realised_returns_5 <- returns_data_4 %>%
  filter(
    Date >= as.Date("2022-01-01"),
    Date <= as.Date("2022-09-30")
  ) %>%
  arrange(Date)

# Select corresponding portfolio weights from Exercise 4
weights_5 <- portfolio_weights_4[1:9, , drop = FALSE]

# Calculate risk-free portfolio
rf_weights_5 <- 1 - rowSums(weights_5)


rf_data_5 <- monthly_cleaned %>%
  arrange(Date) %>%
  mutate(
```

```
    rf_monthly_5 = lag(.data[[rf_col_3]]) / 100 / 12
  )

rf_rates_5 <- rf_data_5$rf_monthly_5[
  match(realised_returns_5$Date, rf_data_5$Date)
]

# Calculate returns
stock_returns_5 <- as.matrix(realised_returns_5[, -1])

portfolio_returns_5 <- rowSums(
  weights_5 * stock_returns_5
) + rf_weights_5 * rf_rates_5

cumulative_return_5 <- (
  prod(1 + portfolio_returns_5) - 1
) * 100

# Create results table
results_5 <- data.frame(
  Month = format(realised_returns_5$Date, "%B %Y"),
  Portfolio_Return = portfolio_returns_5 * 100
)

# Results
print(results_5)
print(cumulative_return_5)
```

# Exercise 6

### Question

(2 points) Calculate the Sharpe ratio for the modern portfolio theory (MPT) portfolio in Q5 using monthly data for January–September 2022. Also calculate mean excess returns and volatilities for the rolling minimum-variance portfolio (estimated as in Q4), the equally weighted portfolio, and the value-weighted portfolio. These three comparison portfolios do not include the risk-free asset. Rebalance the equally weighted portfolio monthly and use uncapped value weights for the value-weighted portfolio.

Calculate their Sharpe ratios for January–September 2022. Which of the four strategies has the highest Sharpe ratio?

### Answer

The Sharpe ratios were calculated using realised monthly returns from January to September 2022. Each portfolio was evaluated based on its mean excess return and sample volatility.

The portfolio with the highest Sharpe ratio is **the equally weighted portfolio**, indicating the best risk-adjusted performance over the sample period.

Therefore, the correct answer is **b. Equally weighted portfolio**.

Table 4: Comparison of Monthly Sharpe Ratios

| Portfolio | Mean Excess Return (%) | Volatility (%) | Sharpe Ratio |
|---|---|---|---|
| MPT | -1.092 | 3.258 | -0.335 |
| Equally Weighted | -2.639 | 8.203 | -0.322 |
| Value Weighted | -2.945 | 8.614 | -0.342 |
| Minimum Variance | -2.304 | 5.563 | -0.414 |

### R Code

```
# Exercise 6

# MPT portfolio from Exercise 5
mpt_returns_6 <- portfolio_returns_5

# Equally weighted portfolio
equal_returns_6 <- rowMeans(stock_returns_5)

# Value-weighted portfolio without 35% cap
market_caps_6 <- as.matrix(
  caps_1[
    match(realised_returns_5$Date, data_1$Date),
  ]
)

value_weights_6 <- market_caps_6 / rowSums(market_caps_6)

value_returns_6 <- rowSums(
  value_weights_6 * stock_returns_5
```

```
)

# Rolling global minimum-variance portfolio
gmv_weights_6 <- matrix(
  NA_real_,
  nrow = nrow(realised_returns_5),
  ncol = length(stocks)
)

gmv_returns_6 <- numeric(nrow(realised_returns_5))

for (i in seq_len(nrow(realised_returns_5))) {

  # Previous 132 monthly returns
  window_6 <- returns_data_4 %>%
    filter(Date < realised_returns_5$Date[i]) %>%
    tail(132)

  # Estimate sample covariance matrix
  Sigma_6 <- cov(window_6[, -1])
  ones_6 <- rep(1, length(stocks))

  # Calculate GMV weights
  weights_6 <- as.numeric(
    solve(Sigma_6, ones_6) /
      sum(solve(Sigma_6, ones_6))
  )

  # Calculate realised portfolio return
  gmv_weights_6[i, ] <- weights_6
  gmv_returns_6[i] <- sum(
    weights_6 * stock_returns_5[i, ]
  )
}

# Combine realised monthly returns
portfolio_comparison_6 <- data.frame(
  MPT = mpt_returns_6,
  Equal_Weighted = equal_returns_6,
  Value_Weighted = value_returns_6,
  Minimum_Variance = gmv_returns_6
)

# Calculate monthly excess returns
excess_returns_6 <- portfolio_comparison_6 - rf_rates_5

# Calculate mean excess returns and sample volatilities
mean_excess_6 <- colMeans(excess_returns_6)
volatility_6 <- sapply(portfolio_comparison_6, sd)

# Calculate monthly Sharpe ratios
sharpe_ratios_6 <- mean_excess_6 / volatility_6
```

```r
# Create comparison table
results_6 <- data.frame(
  Portfolio = c(
    "MPT",
    "Equally Weighted",
    "Value Weighted",
    "Minimum Variance"
  ),
  Mean_Excess_Return = as.numeric(mean_excess_6) * 100,
  Volatility = as.numeric(volatility_6) * 100,
  Sharpe_Ratio = as.numeric(sharpe_ratios_6)
)

# Results
print(results_6)
```